Supplementary Document
Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

This document holds the tables and the figure that the main paper's page budget could not carry. Nothing here introduces a claim the paper does not already state in prose: each item is the tabular or graphical form of a number the Results section gives, provided so a reader can check it against the released per-submission logs rather than against a sentence. Assignment ids (R1–R5, C1–C6) and all metric definitions are the main paper's.

Every value is generated from `results/v2/` by the released scripts (`run.py tables`, `run.py figures`); none is transcribed.

Where the Repairs Come From (RQ3)

Table S1 decomposes every repair by the mechanism that produced it, with the mean patch size each mechanism leaves and whether its patch carries a misconception label. Shares are over repaired submissions under the frontier builder; the fallback row belongs to CoEP_{LF} and sits outside that denominator. The final rows are the framework-only ablation — the identical tool with the LLM-authored rules removed and the generic library and transplant left in place, so exactly one layer differs.

Repair source	share (%)	mean RPS	labelled
1. rules	24.6	0.100	•
2. reference transplant	75.4	0.581	–
3. LLM (CoEP_{LF})	8.2	0.724	–
Framework-only ablation (builder rules removed), first column is RR (%) rather than share			
ablated	98.69	0.492	–
full tool	99.06	0.463	–

Table S1: Repair source, patch size, and diagnosis labelling, with the framework-only ablation. The rule layer is the only one that leaves a misconception label behind; the transplant is what carries coverage.

What the Conclusion Survives (RQ5)

Table S2 gives the three robustness axes side by side: assignment difficulty, the choice of builder, and the escalation threshold. The difficulty bands group the eleven assignments as CS1, Codeforces 800–1000, and Codeforces 1200–2000; the last band folds C6's three submissions in rather than reporting them alone.

Stratum	Refactory	PyDex RR (%) / RPS	CoEP_{LF}
Difficulty (`claude-sonnet-5`)			
CS1	90.8 / 0.397	98.6 / 0.382	99.4 / 0.269
CF 800–1000	83.1 / 0.663	99.4 / 0.942	100.0 / 0.572
CF 1200–2000	54.5 / 0.815	100.0 / 0.800	100.0 / 0.689
Builder			
`claude-sonnet-5`	–	99.0 / 0.598	99.7 / 0.398
`gpt-5.4-mini`	–	98.9 / 0.669	99.7 / 0.411
Threshold, 29 measured settings			
range	–	–	99.7 / 0.378–0.451
spend		$12.47–$15.61 total	

Table S2: Difficulty, builder, and escalation threshold. The symbolic baseline loses 36 points of repair rate between the easiest and hardest bands while CoEP_{LF} does not move; the two builders differ by 3% in patch size for an 18-fold difference in build price; and across 29 measured threshold settings the repair rate is invariant.

Figure S1 plots every one of those threshold settings against the baselines they are compared to. The point of the figure is the emptiness between the cluster and the baselines: the entire family of settings spans $12.47–$15.61 of total spend, so which one is shipped cannot change any conclusion the paper draws. The shipped setting — escalate a passing local patch whose RPS exceeds 1 — is marked with a star and is the canonical CoEP_{LF} run, not the sweep's reconstruction of it.

Rule Provenance

The builder authored 91 rules that fired at least once, against the shared generic library's 5; they fired 789 times against the library's 293. The distinction that matters is reuse. Exactly one authored rule fired on more than one assignment, while `generic_flip_compare` alone fired 198 times across 8 of the 11 assignments, `generic_shift_range` 43 times across 6, and `generic_shift_index` 16 times across 5. The authored layer is assignment-specific by construction,

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

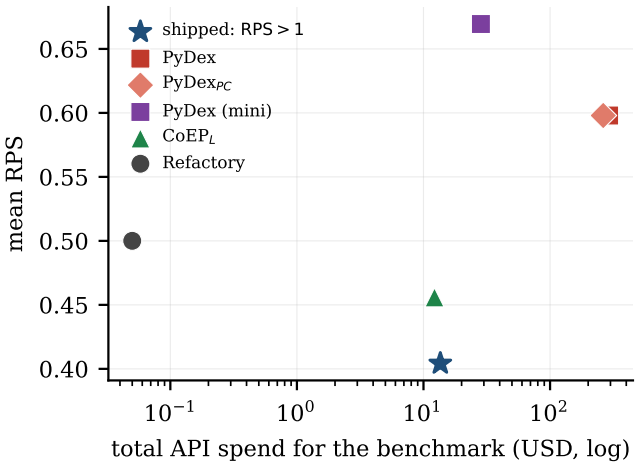


Figure S1: Mean RPS against total API spend for the benchmark. Each small marker is one escalation-threshold setting; the star is the shipped rule. Repair rate is identical across all of them, so this plane is the whole trade-off.

which is why it is priced per assignment and the generic layer is not.

The most-fired authored rules are single-assignment and legible as misconceptions rather than as patches: `slice_case_check_to_equality` (115 firings), `count_compare_relax` (112), `append_missing_return_len_after_loop` (108), `capitalize_title_to_swapcase` (45), and `undefined_domain_var_rename` (28). On C4 a single rule, `carry_fix_two_value_print`, repairs 27 of the assignment's 38 submissions.

The rules self-learning absorbed read the same way: `uniqueness_count_gt1_to_neq1`, `counter_var_confusion`, `index_vs_count_and_uncalled_bool` and `missing_else_false` on R2 with the frontier builder; `count_equality_for_uniqueness`, `count_uniqueness_instead_of_direct_membership` and `missing_call_and_predicate_fix` on R2 with the cheap one; and `binarysearch_zero_exit` on C3.

Released Artifacts

The repository contains the harness, the dataset-import pipeline, every prompt, the synthesized repairers and their build logs, the raw model responses, and the per-submission event logs each arm wrote. Four commands regenerate every number in both documents from those logs: `run.py stats` (Wilson intervals and exact McNemar), `run.py tables`, `run.py numbers`, and `run.py figures`. The 29 measured threshold settings are archived as `results/v2/threshold_sweep.json`.